



Assignment2

Part 1: Node.js Core Modules (10 Grades)

1. Write a function that logs the current file path and directory. (0.5 Grade)
 - Output Example: {File: "/home/user/project/index.js", Dir: "/home/user/project"}
2. Write a function that takes a file path and returns its file name. (0.5 Grade)
 - Input Example: /user/files/report.pdf
 - Output Example: "report.pdf "
3. Write a function that builds a path from an object (0.5 Grade)
 - Input Example: { dir: "/folder", name: "app", ext: ".js" }
 - Output Example: "/folder/app.js"
4. Write a function that returns the file extension from a given file path. (0.5 Grade)
 - Input Example: /docs/readme.md"
 - Output Example: ".md"
5. Write a function that parses a given path and returns its name and ext. (0.5 Grade)
 - Input Example: /home/app/main.js
 - Output Example: { Name: "main", Ext: ".js" }
6. Write a function that checks whether a given path is absolute. (0.5 Grade)
 - Input Example: /home/user/file.txt
 - Output Example: true
7. Write a function that joins multiple segments (0.5 Grade)
 - Input: "src", "components", "App.js"
 - Output Example: src/components/App.js
8. Write a function that resolves a relative path to an absolute one. (0.5 Grade)
 - Input Example: ./index.js
 - Output Example: /home/user/project/src/index.js
9. Write a function that joins two paths. (0.5 Grade)
 - Input Example: /folder1, folder2/file.txt
 - Output Example: /folder1/folder2/file.txt
10. Write a function that deletes a file asynchronously. (0.5 Grade)
 - Input Example: /path/to/file.txt
 - Output Example: The file.txt is deleted.
11. Write a function that creates a folder synchronously. (0.5 Grade)
 - Output Example: "Success"
12. Create an event emitter that listens for a "start" event and logs a welcome message. (0.5 Grade)
 - Output Example: Welcome event triggered!
13. Emit a custom "login" event with a username parameter. (0.5 Grade)
 - Input Example: "Ahmed"
 - Output Example: "User logged in: Ahmed"
14. Read a file synchronously and log its contents. (0.5 Grade)
 - Input Example: "./notes.txt"
 - Output Example: the file content => "This is a note."
15. Write asynchronously to a file. (0.5 Grade)
 - Input: path: "./async.txt", content: "Async save"
16. Check if a directory exists. (0.5 Grade)
 - Input Example: "./notes.txt"
 - Output Example: true
17. Write a function that returns the OS platform and CPU architecture. (0.5 Grade)
 - Output Example: {Platform: "win32", Arch: "x64"}

Assignment2

18. Use a readable stream to read a file in chunks and log each chunk. (0.5 Grade)
- Input Example: `./big.txt`
 - Output Example: log each chunk
19. Use readable and writable streams to copy content from one file to another. (0.5 Grade)
- Input Example: `./source.txt`, `./dest.txt`
 - Output Example: File copied using streams
20. Create a pipeline that reads a file, compresses it, and writes it to another file. (0.5 Grade)
- Input Example: `./data.txt`, `./data.txt.gz`

Part2: Simple CRUD Operations Using HTTP (5 Grades)

- For all the following APIs, you must use the `fs` module to read and write data from a JSON file (e.g., `users.json`).
- Do not store or manage data using arrays

- 1) Create an API that adds a new user to your users stored in a JSON file **(1 Grade)**
(ensure that the email of the new user doesn't exist before)

- URL: `POST /user`

input	output
<pre>{ "name": "User 1", "age": 27, "email": "user@email.com" }</pre>	<pre>{ "message": "User added successfully." }</pre>
<pre>{ "name": "User 2", "age": 30, "email": "user@email.com" }</pre>	<pre>{ "message": "Email already exists." }</pre>

- 2) Create an API that updates an existing user's name, age, or email by their ID. The user ID should be retrieved from the URL **(1 Grade)**

Note: Remember to update the corresponding values in the JSON file

- URL: `PATCH /user/id`

input	output
<pre>{ "age": 30 }</pre>	<pre>{ "message": "User age updated successfully." }</pre>
<pre>/user/99</pre>	<pre>{ "message": "User ID not found." }</pre>

- 3) Create an API that deletes a User by ID. The user id should be retrieved from the URL **(1 Grade)**

Note: Remember to delete the user from the file

- URL: `DELETE /user/id`

input	output
<pre>/user/1</pre>	<pre>{ "message": "User deleted successfully." }</pre>
<pre>/user/99</pre>	<pre>{ "message": "User ID not found." }</pre>

- 4) Create an API that gets all users from the JSON file. **(1 Grade)**

- URL: `GET /user`

input	output
<pre>—</pre>	<pre>[{ "id": 1, "name": "User 1", "age": 27, "email": "user@email.com" }]</pre>

Assignment2

5) Create an API that gets User by ID. (1 Grade)

- **URL:** GET /user/:id
- **Output:**

input	output
/user/1	{ "id": 1, "name": "User 1", "age": 27, "email": "user@email.com" }
/user/99	{ "message": "User not found." }

! important Notes about postman

- 1-Name the endpoint with a meaningful name like 'Add User', not dummy names.
- 2-Save your changes on each request(ctrl+s).
- 3-Include the Postman collection link (export your Postman collection) in the email with your assignment link



Assignment2



Part3: Bonus (3 Grades):

How to deliver the bonus?

- 1- Solve the problem [kth-missing-positive-number](#) on LeetCode
- 2- Inside your assignment folder, create a SEPARATE FILE and name it "bonus.js"
- 3- Copy the code that you have submitted on the website inside "bonus.js" file